

Chapter 82

Take-Home

Assignments & Mock

Interviews

Complete a take-home assignment the way a strong candidate actually would, not just technically correctly, but with the judgment and communication a reviewer is scoring · sit through a full, realistic mock interview and see exactly what a strong versus weak answer sounds like at each turn · read...

Approved chapter · v1

Contents

Chapter 82. Take-Home Assignments & Mock Interviews

- 82.1 Take-home assignment: Data Analyst
- 82.2 Take-home assignment: Data Scientist
- 82.3 Take-home assignment: Data Engineer
- 82.4 Mock interview: Data Analyst
- 82.5 Mock interview: Data Scientist
- 82.6 Mock interview: Data Engineer
- Common mistakes and how to spot them
- Tools
- The project
- Key terms
- Where this leads

Chapter 82. Take-Home Assignments & Mock Interviews

Part VIII — The Interview Playbook

A scope note. The blueprint calls for 6 take-homes and 8 mock scripts across six roles. This chapter delivers **3 complete take-home assignments with real, verified model submissions** and **3 full mock interview scripts with interviewer notes and scoring**, one each for Data Analyst, Data Scientist, and Data Engineer, matching this chat's confirmed DA/DS/DE focus rather than spreading thinner across all six roles the blueprint names. Same depth-over-count trade-off as every other bank in this part.

You will learn to: complete a take-home assignment the way a strong candidate actually would, not just technically correctly, but with the judgment and communication a reviewer is scoring · sit through a full, realistic mock interview and see exactly what a strong versus weak answer sounds like at each turn · read interviewer scoring notes and understand what actually moved the needle.

How this chapter is built. Every take-home's model submission was **actually run**, not written to look plausible: real SQL against the live Riverstone database, real Python, real output. Every mock script is a full, realistic transcript, not a summary, with the candidate's answers labeled against Chapter 69's rubric and interviewer scoring notes at the end.

82.1 Take-home assignment: Data Analyst

The assignment, as given

"Riverstone's sales leadership wants to understand which product categories are driving growth and which need attention. Using the attached order data, prepare a short analysis (a one-page summary plus supporting queries/code) answering: which categories are performing best and worst, what's driving the difference, and one specific recommendation. You have 2 hours."

Model submission

Step 1: the core numbers, verified live:

```
SELECT p.category,
       SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS revenue,
       COUNT(DISTINCT o.order_id) AS orders
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.status <> 'Cancelled'
GROUP BY p.category ORDER BY revenue DESC;
```

category	revenue	orders
Storage	2,297,973.50	142
Kitchen	1,208,030.00	85
Industrial	795,830.00	23
Furniture	33,637.50	2

Step 2: one level deeper, checking whether Industrial's smaller order count is offset by higher value per order. First attempt averaged per order *line*, not per actual order, silently the wrong grain; corrected to aggregate revenue per order first, then average those order totals:

```
WITH order_category_revenue AS (
  SELECT o.order_id, p.category, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)) AS
order_revenue
  FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
  JOIN products p ON oi.product_id = p.product_id
  WHERE o.status <> 'Cancelled'
  GROUP BY o.order_id, p.category
)
SELECT category, ROUND(AVG(order_revenue), 2) AS avg_order_value, COUNT(*) AS n_orders
FROM order_category_revenue GROUP BY category ORDER BY avg_order_value DESC;
```

category	avg_order_value	n_orders
Industrial	34,601.30	23
Furniture	16,818.75	2
Storage	16,182.91	142
Kitchen	14,212.12	85

The one-page summary:

Category performance, this period

Storage is Riverstone's clear revenue leader (₹22.98L, 142 orders), driven by high order volume rather than large individual orders. Industrial is a small but high-value category (₹7.96L from just 23 orders), with an average order value more than double Storage or Kitchen's (₹34,601 vs. ₹14,200 to ₹16,200), suggesting Industrial customers place fewer but substantially larger orders, likely bulk B2B purchases. Furniture's average order value (₹16,819) looks competitive with Storage, but it's based on only 2 orders, far too small a sample to draw any real conclusion from; I'm flagging it explicitly rather than either ranking it seriously or dismissing it.

Recommendation: Industrial's high average order value with low volume suggests real headroom, a modest increase in Industrial customer *acquisition* (not just retention) could have outsized revenue impact given each new order is worth roughly double an average Storage order. I'd recommend sales leadership review whether Industrial has a dedicated outreach motion, and if not, pilot one, checking results against this same category breakdown next quarter.

Caveat: this is one period's data; before committing resources, I'd want to confirm this pattern holds over a longer window, and check whether Industrial's high average order value reflects genuinely more customers or just a few large, possibly non-repeating deals. Furniture needs materially more data before any conclusion at all.

How this would be scored

Dimension	What a strong submission does
Correctness	Numbers reconcile; SQL is real and runs without error
Structure	One-page summary is scannable, leads with the finding, not the method
Depth	Goes one level past the obvious ranking (average order value), not just the headline revenue table
Business judgment	Ends in one specific, proportionate recommendation, not five vague ones
Collaboration/honesty	States the caveat about single-period data explicitly, doesn't overclaim

What a weak submission looks like: the revenue table alone, with a generic "Storage is doing well, Furniture needs improvement" observation and no recommendation, no caveat, no second query digging deeper. Technically not wrong, just thin, exactly Chapter 69's core lesson applied to a full assignment instead of one question.

Learn it in: Chapter 69 (the rubric this scoring table is built from) and Chapter 71 (the SQL patterns used throughout).

82.2 Take-home assignment: Data Scientist

The assignment, as given

"We want to predict which leads are likely to convert, using the attached lead data. Build a model, evaluate it properly, and explain in one page how you'd use it in production, including any limitation you'd flag before it ships. You have 3 hours."

Model submission

Step 1: baseline and tuned comparison, reusing this book's own verified Chapter 37 numbers rather than re-deriving from scratch, since a real prior analysis is more credible than a fresh toy example:

```
logistic regression (test AUC):      0.797
tuned gradient boosting (test AUC):  0.839
```

Step 2: evaluation beyond AUC, per Chapter 39's own confusion matrix at a 0.1 threshold:

```
TP=100  FP=386  FN=46  TN=1693
precision = 0.206    recall = 0.685    F1 = 0.316
```

Step 3: a cost-based threshold, not the default 0.5, per Chapter 39's own worked logic:

Using ₹1,500 as the estimated cost of a sales call and a margin-adjusted value per converted lead, a cost-based threshold identifies 627 leads worth calling, producing an estimated ₹2.45M in protected/generated value against ₹1.08M for a call-everyone baseline, over twice the value for the same team capacity.

The one-page summary:

Model: Gradient-boosted classifier, tuned via cross-validation, test AUC 0.839, a meaningful improvement over a logistic regression baseline (0.797), justifying the added complexity.

Evaluation: At a naive 0.5 threshold, the model is nearly useless (flags almost no leads). At a cost-based threshold reflecting the real cost of a sales call against the value of a converted lead, it identifies 627 leads worth prioritizing, an estimated 2.3x more value protected than calling every lead with the same team capacity.

Production use: Score leads nightly, rank by expected value (probability × estimated deal value), not probability alone, since a large low-probability lead can be worth more than a small high-probability one. Hand the sales team a ranked call list, not a raw score.

Limitation I'd flag before shipping: this model is trained on historical conversion patterns and assumes those patterns hold going forward; if lead sourcing channels or market conditions shift meaningfully, the model should be retrained, not trusted indefinitely on its original training data. I'd also flag that "expected value" ranking depends on a margin assumption that should be revisited with finance, not treated as fixed.

How this would be scored

Dimension	What a strong submission does
Correctness	Real baseline-vs-complex comparison, not just one model tried
Structure	Model choice → evaluation → production plan → limitation, in that order
Depth	Goes past AUC into a cost-based threshold, showing the metric-to-business-decision translation
Business judgment	Ranks by expected value, not probability alone, exactly Chapter 44's own lesson
Collaboration/honesty	States a real limitation unprompted, doesn't oversell the model as a finished, permanent solution

What a weak submission looks like: reports only AUC, recommends a 0.5 threshold with no cost reasoning, and has no limitation section at all, technically functional, missing every dimension beyond raw correctness.

Learn it in: Chapter 74 (this whole submission is built from that chapter's own verified numbers) and Chapter 44 (the expected-value ranking logic).

82.3 Take-home assignment: Data Engineer

The assignment, as given

"Design and implement a small pipeline that loads daily order data into a summary table, safely re-runnable if it fails partway through, with at least one automated data quality check. You have 2 hours."

Model submission

Step 1: the idempotent load, verified live:

```
CREATE TABLE daily_summary (order_date DATE PRIMARY KEY, total_revenue NUMERIC, order_count INT);

INSERT INTO daily_summary
SELECT o.order_date, SUM(oi.quantity*oi.unit_price*(1-oi.discount_pct/100)), COUNT(DISTINCT
o.order_id)
FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status <> 'Cancelled' GROUP BY o.order_date
ON CONFLICT (order_date) DO UPDATE SET
    total_revenue = EXCLUDED.total_revenue,
    order_count = EXCLUDED.order_count;
```

Step 2: a data quality check, run immediately after the load, verified live:

```
SELECT COUNT(*) FROM daily_summary WHERE total_revenue < 0;
```

```
count: 0  -- passes; would fail the pipeline run loudly if this ever returned > 0
```

Step 3: the one-page design note:

Idempotency: the load uses `ON CONFLICT ... DO UPDATE`, keyed on `order_date`, so re-running the exact same load after a failure updates existing rows instead of duplicating them, safe to retry with no manual cleanup.

Data quality: an automated check asserts `total_revenue` is never negative, run immediately after every load; if it fails, the pipeline should fail loudly (alert, don't silently proceed), per Chapter 77's monitoring discipline. In a real deployment I'd add a second check for a volume anomaly (today's order count far below the trailing average), since a negative-revenue check alone wouldn't catch a source system silently sending a truncated file.

What I'd add given more time: a `<!-- db -->`-style staging layer so raw extracted data lands unmodified before transformation, letting a transformation bug be fixed and reprocessed without re-extracting from the source; and an explicit dependency declaration if this joins with other scheduled loads, rather than assuming execution order.

How this would be scored

Dimension	What a strong submission does
Correctness	The upsert genuinely works, re-running it doesn't duplicate rows
Structure	Idempotency, quality check, and design note clearly separated, not tangled together
Depth	Names a <i>second</i> check type unprompted (volume anomaly), not just the one requested
Business judgment	Explains <i>why</i> a pipeline should fail loudly rather than silently on a bad check
Collaboration/honesty	States what's missing given more time, rather than presenting the 2-hour version as fully production-ready

What a weak submission looks like: a plain `INSERT` with no conflict handling (fails the core "safely re-runnable" requirement entirely), or a quality check that only logs a warning instead of failing the run.

Learn it in: Chapter 77 (every pattern in this submission is drawn directly from that chapter's own verified examples).

82.4 Mock interview: Data Analyst

A full 20-minute mock, condensed. Interviewer notes appear in brackets; scoring appears at the end.

Interviewer: Let's start with something practical. Write a query to find customers who haven't placed an order in the last 90 days.

Candidate: Sure. Quick check first, when you say "last 90 days," is that from today, or from the most recent order date in the dataset? Since if this is historical data, "today" might not mean what I'd assume. *[Interviewer note: good, caught a real ambiguity most candidates miss.]*

Interviewer: Good catch. Assume from the most recent date in the data.

Candidate: Got it.

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE c.customer_id NOT IN (
  SELECT customer_id FROM orders
  WHERE order_date > (SELECT MAX(order_date) FROM orders) - INTERVAL '90 days'
  AND customer_id IS NOT NULL
);
```

I used `NOT IN` here, but I want to flag, if that subquery could ever return a `NULL` `customer_id`, this breaks silently, returns zero rows. I added the explicit `NOT NULL` filter for exactly that reason, but I'd actually prefer a `NOT EXISTS` version to avoid the risk entirely. *[Interviewer note: strong, unprompted; knows the classic trap and pre-empted it rather than needing to be caught.]*

Interviewer: Let's say this returns 40 customers. Sales wants to know which to call first. How would you prioritize?

Candidate: I'd rank by revenue at risk, not just recency, probability of churn times their historical value, if I have that, or at minimum just their historical revenue as a proxy if I don't have a model. Ranking by "most overdue" alone would undervalue a large account that's only slightly overdue against a small one that's very overdue. *[Interviewer note: connects a simple query to a real business prioritization insight without being asked to.]*

Interviewer: Last one. Walk me through a project where an analysis you did led to an actual decision.

Candidate: *(gives a full STAR-format answer, ~90 seconds, ending with a quantified result)* *[Interviewer note: quantified, specific, ends with impact, not just method.]*

Scoring

Dimension	Score (1-4)	Why
Correctness	4	Query correct, NULL trap explicitly addressed
Structure	4	Clarified before answering, signposted throughout
Depth	4	Volunteered the prioritization insight and the NOT EXISTS preference unprompted
Business judgment	4	Revenue-at-risk framing, not just "most overdue"
Collaboration	4	Asked a genuinely clarifying question early, engaged naturally throughout

Overall: strong pass. This is what a 4-across-every-dimension answer actually sounds like, not flawless in a dramatic way, just consistently a step past "correct" on every single turn.

82.5 Mock interview: Data Scientist

A full 25-minute mock, condensed.

Interviewer: Tell me about a model you're proud of.

Candidate: *(gives the full Chapter 76A-style project narrative: framing, baseline-vs-complex comparison, cost-based threshold, honest limitation)* *[Interviewer note: hits every structural element without being prompted for each one individually.]*

Interviewer: You mentioned AUC of 0.839. Walk me through what that number actually means, and whether it's "good."

Candidate: AUC is the probability that the model ranks a randomly chosen positive case above a randomly chosen negative one. 0.839 means it's doing that correctly about 84% of the time, which is solid but not exceptional, whether it's "good enough" really depends on what decision it's feeding into, not the number in isolation. For this specific case, pairing it with a cost-based threshold mattered more than the AUC number itself for whether it was actually useful. *[Interviewer note: correct technical definition, and resists treating the number as inherently meaningful without business context.]*

Interviewer: What would you do if a stakeholder asked you to make the model "more accurate" with no further detail?

Candidate: I'd ask what's actually driving that request, is a specific kind of error causing a real problem, missed high-value leads, too many false positives wasting sales time, something else? "More accurate" could mean improving precision, recall, or overall AUC, and those often trade off against each other, so I'd want to know which error actually matters more before doing anything. *[Interviewer note: same diagnostic-before-acting instinct as Chapter 76B's "make the report better" question, applied here to a model instead of a report.]*

Interviewer: Your model's feature importance shows a customer ID column ranking unusually high. What's your first reaction?

Candidate: That's a red flag, not a good sign. An ID shouldn't be genuinely predictive on its own; if it looks like it is, I'd suspect either a leak, maybe IDs were assigned in a way that correlates with signup cohort or another leaked signal, or a bug in how the data was split, like an unshuffled time-based split letting row order leak information. I'd check that before trusting anything else about the model. *[Interviewer note: immediately named leakage as the concern, gave two specific real mechanisms, not just "that's suspicious."]*

Scoring

Dimension	Score (1-4)	Why
Correctness	4	Technically accurate AUC definition, correct leakage diagnosis
Structure	4	Full project narrative hit every structural beat unprompted
Depth	4	Named two specific leakage mechanisms, not a vague "that's odd"
Business judgment	4	Explicitly tied AUC's meaning to the decision it feeds, resisted treating it as inherently meaningful
Collaboration	3	Strong throughout; could have asked one more clarifying question on the "more accurate" request before answering as fully as they did

Overall: strong pass, with one small, realistic note (even a very strong candidate doesn't need a perfect 4 everywhere to pass well).

82.6 Mock interview: Data Engineer

A full 25-minute mock, condensed.

Interviewer: Walk me through how you'd design a pipeline that loads daily sales data, and make it safe to re-run if it fails.

Candidate: *(gives the full staged design: extract, land raw, transform, idempotent upsert load, quality checks, orchestration, monitoring, closely matching this chapter's own §82.3 model submission)*
[Interviewer note: complete, staged answer, didn't just describe the load step.]

Interviewer: Your pipeline ran successfully last night, no errors, but this morning the numbers look wrong. Walk me through debugging this.

Candidate: First I'd separate two possibilities: did it actually process the right data and compute something wrong, or did it succeed technically while processing something incomplete, like a source file that arrived truncated. I'd check the row counts it processed against a trailing average first, since a silent volume drop is a common cause of "ran fine, wrong numbers" that a pure success/failure check wouldn't catch. If the volume looks normal, I'd trace the actual transformation logic for a recent change. *[Interviewer note: immediately reached for the volume-anomaly check as the first diagnostic step, not a random guess.]*

Interviewer: Say you find the issue: a source column got renamed upstream, and your pipeline was silently defaulting the missing column to zero instead of erroring. What do you change, beyond fixing this one instance?

Candidate: Two things, not just the immediate fix. I'd change the load to fail loudly if an expected column is missing, rather than defaulting silently, that's the actual root cause, not the rename itself. And I'd add a validation step that checks the pipeline's own output against a rough expected range before it's considered successful, so a similarly silent wrong-result failure gets caught the same day, not whenever someone happens to notice the numbers look off. *[Interviewer note: distinguished the specific bug from the systemic gap that let it go unnoticed, exactly the depth this kind of question is fishing for.]*

Interviewer: How would you decide whether this pipeline should be batch or streaming?

Candidate: By how fresh the actual downstream decision needs to be, not by which is more modern. A daily sales summary report doesn't need sub-minute freshness, so batch is the right, simpler choice here. I'd only reach for streaming if there were a genuine decision that couldn't wait for the next batch window. *[Interviewer note: correctly resisted the instinct to recommend streaming as the more impressive answer.]*

Scoring

Dimension	Score (1-4)	Why
Correctness	4	Idempotent design correct, batch-vs-streaming reasoning correct
Structure	4	Full staged pipeline design, not just the interesting part
Depth	4	Volume-anomaly check as first diagnostic instinct, not a guess
Business judgment	4	Distinguished the specific bug from the systemic monitoring gap
Collaboration	4	Engaged naturally with each follow-up, no defensiveness under the "why did this go wrong" question

Overall: strong pass, a clean sweep, worth noticing that it's still built from concrete, specific answers at every turn, not confidence alone.

Common mistakes and how to spot them

Mistake	Symptom	Fix
A take-home with no summary, just code and output	Reviewer has to reverse-engineer the finding themselves	Always include a one-page summary leading with the answer
A take-home with no stated limitation	Reads as overconfident or unaware of real gaps	State at least one honest limitation, even under time pressure
A mock answer that's technically correct but thin	Passes on correctness, scores low on depth and business judgment	Practice adding at least one unprompted extra-point move per answer
Treating every mock question as isolated	Missed opportunities to connect answers into a coherent narrative	Notice when a later question connects to an earlier answer, and say so
Defensiveness under a "why did this go wrong" follow-up	Reads as fragility, costs the collaboration dimension	Engage with the question directly, as genuine problem-solving, not a personal challenge

Tools

PostgreSQL 16 and **Python**, the same environment used throughout this part; every SQL and data quality claim in this chapter's take-homes was actually run against live Riverstone data. A timer, for practicing take-homes and mock answers under the same time pressure a real interview or assignment actually imposes.

The project

Goal: complete one of this chapter's three take-homes yourself, from scratch, before reading the model submission again.

1. Set a timer matching the stated time limit, and attempt the assignment cold.
2. Compare your submission against this chapter's scoring table, dimension by dimension, not just "was I right."
3. Record yourself doing one of the three mock interviews, playing both roles if needed, and compare your answers against the scored transcript.
4. Identify the one dimension (correctness, structure, depth, business judgment, collaboration) you scored lowest on, and deliberately practice that one on your next three practice questions from any earlier chapter in this part.

Key terms

take-home assignment · model submission · mock interview · interviewer scoring notes · one-page summary (take-home) · cost-based threshold (reused) · idempotent load (reused) · volume-anomaly check (reused)

Where this leads

- **Chapter 69, The Extra-Points Method**, is the rubric every scoring table in this chapter is built from.
- **Every other chapter in Part VIII** supplied the actual content these take-homes and mocks draw on: Chapter 71 (SQL), Chapter 74 (ML evaluation), Chapter 77 (pipeline design), Chapter 44 (expected-value ranking), Chapter 76A/76B (project narrative structure).
- This is the last chapter in Part VIII, The Interview Playbook. The reader who's worked through this part end to end has, in effect, already sat through the mocks in this chapter once, for real, one question at a time.